



**INSTITUTO POLITECNICO NACIONAL
ESCUELA SUPERIOR DE COMPUTO**



**BASES DE DATOS
PRACTICA No.1
“Modelo Entidad-Relación”**

ALUMNOS:

Daniel Patiño Farias
Arath Sebastian Sanchez Valenzuela
Alejandro Rivas Ramirez

PROFESOR:

Gabriel Hurtado Avilés

GRUPO:

3CV4

Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación:

1. Un **sistema de control de versiones (VCS)** es un software que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo. Lo que hace es permitir que varias personas trabajen en el mismo proyecto sin cambiar el trabajo de los demás. El problema concreto que resuelve en un trabajo en equipo es el caos de la sobreescritura y la pérdida de información. Si dos personas editaban el mismo archivo al mismo tiempo, el último en guardar borraba los cambios del primero, un VCS elimina esto al fusionar los cambios de forma inteligente y mantener un historial completo donde se puede saber quién hizo qué, cuándo y por qué, permitiendo regresar a cualquier punto del pasado si algo sale mal.

2. **Git** es el sistema de control de versiones en sí, es una herramienta de código abierto que se instala localmente en la computadora y funciona mediante la línea de comandos para registrar el historial de cambios. **GitHub** es una plataforma en la nube (un servicio) que guarda los proyectos gestionados con Git. Git es el motor que rastrea los archivos, mientras que GitHub es el sitio web que permite compartir esos archivos con todos, colaborar visualmente con otros programadores y respaldar el trabajo en internet.

3. Para entender el ecosistema de Git, es necesario dominar sus conceptos clave:

- **Repositorio:** Es la carpeta digital donde se guarda todo tu proyecto, incluyendo el historial de cambios. *Ejemplo:* Una carpeta llamada "MiAplicacion" donde vive todo el código.
- **Confirmación (Commit):** Es una captura del estado de tus archivos en un momento específico, como guardar una partida en un videojuego. *Ejemplo:* Guardar los cambios tras diseñar la pantalla de inicio y escribir el mensaje "Se añade diseño de login".
- **Rama (Branch):** Es una línea de tiempo independiente para desarrollar funciones sin alterar el código principal.
- **Fusión (Merge):** El proceso de unir dos ramas para integrar los cambios de una en la otra. *Ejemplo:* Llevar los cambios de la rama cambiar-colores de vuelta a la rama main una vez que el diseño oscuro quedó perfecto.
- **Conflicto de fusión:** Ocurre cuando dos personas modifican la misma línea de un archivo en ramas distintas y Git no sabe cuál versión conservar.
- **Pull Request:** Es aviso en GitHub para avisar al equipo que has terminado una tarea y quieres fusionar tu trabajo con el principal.
- **Archivo .gitignore:** Un archivo de texto que le dice a Git qué carpetas o archivos debe ignorar por completo.
- **Archivo README:** El documento de presentación de tu proyecto. *Ejemplo:* Un archivo de texto donde escribes de qué trata el proyecto, cómo se instala y los pasos para ejecutarla.

4. Finalmente, el **flujo de trabajo basado en ramas** consiste en mantener una rama principal siempre limpia y funcional (normalmente llamada main o master), mientras que

cada nueva característica, corrección de error o experimento se desarrolla en su propia rama aislada.

El código se revisa entre pares antes de fusionarse por razones de calidad, seguridad y aprendizaje. Esto hace que otros programadores detecten errores, sugieran mejoras o identifiquen fallas de seguridad antes de que el nuevo código afecte a los usuarios finales.

Ejercicio 2. El sistema gestor en un contenedor: Docker

Parte A. Investigación.

Un **contenedor** es un entorno aislado que permite ejecutar una aplicación junto con los archivos, bibliotecas y dependencias que necesita para funcionar. En Docker, los contenedores se crean a partir de imágenes y permiten que las aplicaciones se ejecuten de manera más uniforme en diferentes equipos. A diferencia de una máquina virtual, que incluye un sistema operativo completo con su propio núcleo, los contenedores comparten el núcleo del sistema operativo del equipo anfitrión. Por esta razón, generalmente requieren menos recursos y pueden iniciar más rápidamente. En cuanto al aislamiento, ambos proporcionan separación entre aplicaciones, pero las máquinas virtuales ofrecen un aislamiento basado en la virtualización del hardware, mientras que los contenedores aíslan procesos a nivel del sistema operativo. Dentro de este ecosistema, se definen los siguientes componentes clave:

Imagen: Un mapa de bits o plantilla de solo lectura que contiene las instrucciones para crear un contenedor.

Contenedor: La instancia viva y ejecutable de una imagen.

Volumen: Un mecanismo de almacenamiento persistente administrado por el motor de contenedores que reside fuera del sistema de archivos de este.

Puerto publicado: Un mapeo que expone un puerto del contenedor hacia un puerto de la máquina host, permitiendo el tráfico de red externo hacia la aplicación.

En Bases de Datos, el **volumen** es indispensable debido a lo pequeño de los contenedores. El ciclo de vida de un contenedor está diseñado para que este pueda destruirse, recrearse o actualizarse sin previo aviso, si no se declara un volumen, todos los datos generados por el sistema gestor de bases de datos se escribirán en la capa de almacenamiento de lectura/escritura propia del contenedor. Al detenerse o eliminarse el contenedor, toda la información y las bases de datos creadas se perderán de forma irreversible, ya que dicha capa se destruye junto con él. El volumen garantiza que los datos persistan de manera segura e independiente en el host informático.